

Diseño y desarrollo de una arquitectura multiagente
basada en LangGraph
para procesamiento de datos y detección de
anomalías industriales

Autor: pendiente

Curso académico: pendiente

Resumen

Este Trabajo Fin de Máster propone una arquitectura multiagente para el procesamiento, modelado y detección de anomalías en entornos industriales. El caso de uso inicial se centra en rodamientos de motores eléctricos monitorizados mediante señales de vibración. La solución combina agentes coordinados mediante LangGraph con ejecutores Python deterministas, de forma que las decisiones se representen mediante contratos estructurados y las transformaciones sobre datos sean reproducibles, audita-
bles y testeables.

Abstract

This Master's Thesis proposes a multi-agent architecture for data processing, modeling and anomaly detection in industrial environments. The initial use case focuses on electric motor bearings monitored through vibration signals. The solution combines agents coordinated with LangGraph and deterministic Python executors, so that decisions are represented through structured contracts and data transformations remain reproducible, auditable and testable.

Índice general

1. Introducción	1
1.1. Contexto	1
1.2. Motivacion	1
1.3. Problema	1
1.4. Objetivo general	2
1.5. Objetivos específicos	2
1.6. Alcance y limitaciones iniciales	2
2. Estado del arte	3
2.1. Mantenimiento predictivo y detección de anomalías	3
2.2. Modelos de detección de anomalías	3
2.3. Sistemas multiagente aplicados a pipelines de datos	3
2.4. Datasets públicos de referencia	4
3. Arquitectura propuesta	5
3.1. Principios de diseño	5
3.2. Separación entre agentes y ejecutores	5
3.3. Nodos previstos	5
3.4. Estado global	6
3.5. Contratos estructurados	6
3.6. Trazabilidad	7
4. Metodología	8
4.1. Caso de uso industrial	8
4.2. Pipeline de datos	8
4.3. Manifiesto del dataset	8
4.4. Extracción y perfilado	9
4.5. Estructuración temporal	9
4.6. Particiones	9
4.7. Métricas	9
5. Implementación	10
5.1. Estructura del repositorio	10
5.2. Estado Actual	10
5.3. Estado global implementado	10
5.4. Contratos Pydantic implementados	11
5.5. Artefactos previstos	11
5.6. Adaptadores de Entrada	11
5.7. Ejecutor de manifiesto	12

5.8. Ejecutor de perfilado	12
5.9. Ejecutor de Limpieza	12
5.10. Ejecutor de Estructuración Temporal	13
5.11. Ejecutor de Modelado Base	13
5.12. Ejecutor de Evaluación	13
5.13. Grafo LangGraph supervisado	14
5.14. Agente Limpiador LLM	15
5.15. Agente Estructurador LLM	15
5.16. Agente Modelador LLM	16
5.17. Agente Evaluador LLM	17
5.18. Agente Redactor LLM	17
5.19. Persistencia Local de Ejecuciones	18
5.20. Wrapper de Ejecucion Persistida	19
5.21. Registro Consultable de Ejecuciones	19
5.22. Protocolo Experimental Local	19
5.23. API Minima de Consulta	20
5.24. Dependencias	20
5.25. Verificación Actual	21
5.26. Siguiente Paso de Implementación	21
6. Experimentos	22
6.1. Protocolo experimental	22
6.2. Plan CWRU Isolation Forest Threshold	22
6.3. Experimentos posteriores	23
7. Resultados	24
7.1. Resultados cuantitativos	24
7.2. Análisis cualitativo	24
7.3. Limitaciones observadas	24
8. Conclusiones	26
8.1. Conclusiones principales	26
8.2. Trabajo futuro	26

Índice de figuras

Índice de tablas

6.1. Comparacion local de umbral con Isolation Forest sobre CWRU	23
7.1. Comparacion de umbral con Isolation Forest en el split de test CWRU.	24

Capítulo 1

Introducción

1.1. Contexto

La detección de anomalías en maquinaria industrial es una tarea relevante para mejorar la fiabilidad, reducir paradas no planificadas y apoyar estrategias de mantenimiento predictivo. En sistemas rotativos, como motores eléctricos, turbinas, bombas o rodamientos, las señales de vibración permiten observar cambios de comportamiento asociados a desgaste, desbalanceo, defectos mecánicos o degradación progresiva.

Este trabajo se centra inicialmente en rodamientos de motores eléctricos, usando como dataset principal CWRU Bearing Dataset [2]. El dataset NASA IMS Bearing se conserva como fuente futura para validar escenarios de degradación run-to-failure [4].

1.2. Motivación

El interés del proyecto no se limita a entrenar un modelo de detección de anomalías. El objetivo es construir una arquitectura capaz de coordinar el flujo completo de datos: ingestión, perfilado, limpieza, estructuración temporal, entrenamiento, evaluación, auditoría e informe técnico. Para evitar transformaciones opacas o no reproducibles, los agentes proponen decisiones estructuradas y los ejecutores deterministas aplican las operaciones reales.

1.3. Problema

El problema abordado es la detección automática de patrones anómalos en señales de vibración procedentes de rodamientos. En el MVP se formulará como una tarea binaria:

- clase normal: operación sin fallo;
- clase anómala: cualquier fallo de rodamiento disponible en el dataset.

La clasificación por tipo de fallo se mantiene como extensión posterior. Esta decisión reduce el alcance inicial y permite validar primero la trazabilidad del pipeline multiagente.

1.4. Objetivo general

Diseñar e implementar una arquitectura multiagente basada en LangGraph para procesar datos industriales de vibración y detectar anomalías de forma reproducible, auditable y extensible.

1.5. Objetivos específicos

1. Definir una estructura de proyecto que separe código, memoria académica y recursos.
2. Seleccionar un caso de uso industrial y datasets públicos adecuados.
3. Diseñar un pipeline de datos reproducible para señales de vibración.
4. Definir contratos estructurados para agentes y ejecutores.
5. Implementar ejecutores deterministas para perfilado, limpieza, estructuración temporal, modelado y evaluación.
6. Construir un MVP local antes de abordar Docker, SLURM o frontend.
7. Generar informes técnicos trazables a partir de métricas y artefactos.

1.6. Alcance y limitaciones iniciales

El alcance inicial queda limitado a ejecución local, ficheros crudos en disco y un dataset principal. No se abordarán en el MVP la ejecución distribuida, streaming industrial, integración SLURM, frontend ni persistencia avanzada. NASA IMS se mantiene como extensión experimental una vez validado el flujo sobre CWRU.

Capítulo 2

Estado del arte

2.1. Mantenimiento predictivo y detección de anomalías

Esta sección desarrollará el contexto de mantenimiento predictivo, monitorización de condición y detección de anomalías en maquinaria rotativa.

Aspectos a cubrir:

- diferencias entre mantenimiento correctivo, preventivo y predictivo;
- papel de las señales de vibración en rodamientos y motores;
- fallos comunes: pista interior, pista exterior, bola y desbalanceo;
- diferencias entre detección de anomalías, diagnóstico y pronóstico.

2.2. Modelos de detección de anomalías

El MVP comparará primero modelos clásicos y reproducibles:

- Isolation Forest;
- One-Class SVM;
- Local Outlier Factor;
- PCA con error de reconstrucción.

Los autoencoders densos y modelos recurrentes se considerarán en fases posteriores, cuando el pipeline de tensores esté validado.

2.3. Sistemas multiagente aplicados a pipelines de datos

Esta sección describirá el uso de arquitecturas multiagente para dividir responsabilidades entre nodos especializados. El enfoque del proyecto separa:

- decisiones de alto nivel tomadas por agentes;
- validación de contratos mediante esquemas estrictos;
- ejecución determinista de transformaciones sobre datos;
- auditoría de rutas, configuraciones, métricas y artefactos.

2.4. Datasets públicos de referencia

El dataset principal será CWRU Bearing Dataset [2, 3, 1]. NASA IMS Bearing Dataset se reserva como extensión para escenarios de degradación temporal [4, 5].

Capítulo 3

Arquitectura propuesta

3.1. Principios de diseño

La arquitectura se basa en cuatro principios:

1. separar código, memoria académica y recursos;
2. mantener los datos crudos inmutables;
3. representar decisiones mediante contratos estructurados;
4. ejecutar transformaciones mediante módulos Python deterministas.

3.2. Separación entre agentes y ejecutores

Los agentes no escriben ni ejecutan código arbitrario sobre los datos. Su papel es proponer configuraciones, interpretar perfiles y decidir la siguiente fase del flujo. Los ejecutores leen artefactos desde disco, aplican operaciones deterministas y devuelven rutas, métricas y logs.

3.3. Nodos previstos

El grafo mínimo previsto contiene:

- supervisor;
- agente limpiador;
- ejecutor de limpieza;
- agente estructurador;
- ejecutor de estructuración;
- agente modelador;
- ejecutor de modelado;
- evaluador;

- redactor;
- revisión humana para trabajos costosos.

3.4. Estado global

El estado global de LangGraph no almacenará datasets completos. Debe contener rutas, resúmenes y artefactos:

- identificador de ejecución;
- contexto del proyecto;
- rutas de datos crudos, perfiles, datos limpios, tensores e informes;
- configuraciones de limpieza, estructuración y modelado;
- métricas y evaluación;
- errores, aprobaciones humanas y artefactos generados.

En la implementación inicial se ha definido un modelo canónico ‘TFMStateModel’ validado con Pydantic y una vista ‘TFMState’ basada en ‘TypedDict’ para su uso en LangGraph. Esta separación permite validar contratos de forma estricta antes de entregar el estado al grafo y, al mismo tiempo, mantener una representación ligera y serializable.

El estado inicial del caso CWRU queda fijado en la fase ‘dataset_manifest’, con el nodo siguiente ‘manifest_executor’. El directorio crudo se registra como `codigo/data/raw/cwru_bearing/mat`, el canal principal es ‘DE_time’ y la frecuencia objetivo es 12 kHz. Las rutas de artefactos posteriores permanecen como valores nulos hasta que los ejecutores correspondientes las generen.

3.5. Contratos estructurados

Los contratos iniciales incluyen:

- ‘ProjectContext’, que define dominio, máquina, dataset, objetivo, canal principal y frecuencia objetivo;
- ‘DatasetManifestRow’ y ‘DatasetManifest’, que formalizan el inventario de archivos crudos y sus etiquetas;
- ‘DatasetProfileSummary’, que resume el perfil sin almacenar señales;
- ‘CleaningConfig’, ‘StructuringConfig’ y ‘ModelingConfig’, que fijan las configuraciones que podrán proponer los agentes;
- ‘SupervisorDecision’, ‘CleaningDecision’, ‘StructuringDecision’, ‘ModelingDecision’, ‘EvaluationDecision’ y ‘ReportDecision’, que estructuran las respuestas de los agentes;

- ‘ExecutorResult’ y resultados especializados, que normalizan las salidas de los ejecutores deterministas;
- ‘MetricsReport’ y ‘EvaluationResult’, que estructuran la evaluación;
- ‘PipelineError’, ‘HumanApproval’ y ‘ArtifactRef’, que mantienen trazabilidad de errores, revisiones y artefactos.

Estos esquemas aplican validación estricta y rechazan campos no declarados. Por ejemplo, una fila normal del manifiesto debe declarar los campos de fallo como nulos, mientras que una fila con fallo debe incluir tipo de fallo y diámetro. De forma análoga, un resultado de ejecutor con estado fallido debe incluir un error estructurado, y una decisión no terminal del supervisor debe indicar el siguiente nodo del grafo.

3.6. Trazabilidad

Cada ejecución debe permitir responder:

- qué datos se usaron;
- qué configuración fue validada;
- que ejecutor produjo cada artefacto;
- qué métricas se obtuvieron;
- qué limitaciones permanecen.

Capítulo 4

Metodología

4.1. Caso de uso industrial

El caso de uso seleccionado es la detección de anomalías en rodamientos de motores eléctricos mediante señales de vibración. La elección permite trabajar con datos industriales públicos, etiquetas interpretables y un flujo temporal adecuado para ventanas, features y tensores.

4.2. Pipeline de datos

El flujo metodológico para el MVP queda definido como:

```
datos .mat crudos
-> manifiesto del dataset
-> extracción de señales
-> normalización de frecuencia de muestreo
-> perfilado estadístico
-> limpieza determinista
-> ventanas temporales
-> features y tensores
-> particiones train/validation/test
-> modelos de anomalías
-> evaluación
-> informe técnico
```

4.3. Manifiesto del dataset

El primer artefacto metodológico será un manifiesto con una fila por fichero crudo. Este manifiesto conectará los ‘.mat’ originales con el resto del pipeline. Las columnas previstas son:

Campo	Descripción
file_id	identificador del fichero original
dataset	nombre del dataset
source_path	ruta del fichero crudo
label	clase ‘normal’ o ‘fault’

fault_type	tipo de fallo si existe
fault_diameter_inch	diámetro del defecto
load_hp	carga del motor
rpm	régimen de giro
sensor_channel	canal de sensor usado
source_sample_rate_hz	frecuencia de muestreo original
target_sample_rate_hz	frecuencia objetivo del MVP
source_format	formato original
notes	observaciones

4.4. Extracción y perfilado

El canal principal inicial será ‘DE_time’, asociado al acelerómetro del lado drive-end. El perfilado generará resúmenes estadísticos por fichero y por etiqueta, incluyendo número de muestras, canales detectados, valores no finitos, media, desviación típica, RMS, curtosis, factor de cresta y energía.

4.5. Estructuración temporal

La frecuencia objetivo inicial será 12 kHz. Las ventanas candidatas serán de 2048 o 4096 muestras con solapamiento del 50 %. Esta decisión permite comparar features tabulares con modelos clásicos y conservar ventanas crudas para autoencoders posteriores.

4.6. Particiones

Para evitar fuga de información entre ventanas, no se usará un split aleatorio ingenuo. La estrategia inicial será:

- entrenamiento con ventanas normales;
- validación con ventanas normales retenidas;
- test con ventanas normales retenidas y ventanas con fallo.

4.7. Métricas

Las métricas iniciales serán precision, recall, F1-score, ROC-AUC, PR-AUC, tasa de falsos positivos y matriz de confusión. Desde el punto de vista industrial, el criterio principal será mantener alta sensibilidad ante fallos sin disparar en exceso los falsos positivos.

Capítulo 5

Implementación

5.1. Estructura del repositorio

La implementación se organiza en tres áreas:

- ‘codigo/’: aplicación, tests, datos, modelos, informes y documentación técnica;
- ‘memoria/’: documento LaTeX del TFM;
- ‘recursos/’: PDFs, referencias y diagramas auxiliares.

5.2. Estado Actual

Hasta este punto se ha creado la estructura inicial de directorios, se han descargado los datasets crudos, se ha definido el flujo de datos del MVP y se han implementado el contrato inicial del estado global del grafo y los esquemas Pydantic de entradas, decisiones y resultados. Los datos crudos se conservan en ‘codigo/data/raw/’, junto con un README de trazabilidad.

5.3. Estado global implementado

El estado global se ha dividido en dos capas:

- ‘codigo/app/schemas/state.py’: modelos Pydantic estrictos para validar el estado, configuraciones, métricas, errores y artefactos;
- ‘codigo/app/graph/state.py’: ‘TypedDict’ usado por LangGraph, validador de estado e inicializador del caso CWRU.

El inicializador ‘create_initial_cwru_state’ crea una ejecución preparada para generar el manifiesto del dataset. El estado resultante no contiene señales ni arrays, solo rutas, contexto experimental y campos reservados para artefactos futuros.

5.4. Contratos Pydantic implementados

Se han añadido tres grupos de contratos:

- ‘codigo/app/schemas/dataset.py’: manifiesto del dataset, canales y metadatos de fallo;
- ‘codigo/app/schemas/agent_decisions.py’: decisiones estructuradas del supervisor, limpiador, estructurador, modelador, evaluador y redactor;
- ‘codigo/app/schemas/executor_results.py’: resultados comunes y especializados de ejecutores deterministas.

Estos contratos consolidan la regla arquitectónica de que los agentes producen configuraciones o decisiones validables, mientras que los ejecutores devuelven rutas, métricas, errores y referencias a artefactos, pero no datos pesados.

5.5. Artefactos previstos

- ‘codigo/data/interim/cwru_bearing/manifest.csv’;
- ‘codigo/data/interim/cwru_bearing/profile.json’;
- ‘codigo/data/processed/cwru_bearing/clean_signals/’;
- ‘codigo/data/tensors/cwru_bearing/windows_features.csv’;
- ‘codigo/data/tensors/cwru_bearing/windows_raw.npz’;
- ‘codigo/data/tensors/cwru_bearing/splits.json’;
- ‘codigo/reports/cwru_bearing/’;
- ‘codigo/models/cwru_bearing/’.

5.6. Adaptadores de Entrada

Para reducir el acoplamiento al formato CWRU se ha introducido una capa ligera de adaptadores en ‘codigo/app/services/signal_adapters.py’. Esta capa expone dos operaciones comunes: leer todos los canales numéricos de un fichero y cargar un canal concreto como vector. Los ejecutores de perfilado y limpieza usan ahora esta frontera en lugar de leer directamente ficheros ‘.mat’.

La implementación inicial soporta ‘.mat’, ‘.csv’, ‘.txt’, ‘.tsv’ y ‘.npz’. En el caso de CWRU, el adaptador reconoce los canales ‘DE_time’, ‘FE_time’, ‘BA_time’ y ‘RPM’; en formatos tabulares, interpreta columnas numéricas como canales. Para incorporar un nuevo origen, como NASA IMS, el objetivo será crear o ampliar un adaptador de entrada y mantener estable el resto del pipeline: perfilado, limpieza, estructuración, modelado y evaluación.

5.7. Ejecutor de manifiesto

Se ha implementado el primer ejecutor determinista del pipeline, el módulo ‘data-set_manifest.py’. Su responsabilidad es generar el artefacto:

```
codigo/data/interim/cwru_bearing/manifest.csv
```

a partir de los ficheros ‘.mat’ crudos de CWRU.

El ejecutor no carga señales completas. Solo inspecciona los nombres de fichero y aplica un mapeo controlado de metadatos CWRU: carga, RPM, tipo de fallo, diámetro y posición del fallo exterior cuando corresponde. El manifiesto generado conserva la frecuencia original de cada fichero y fija una frecuencia objetivo de 12 kHz para el MVP.

La ejecución real sobre los datos descargados ha generado 64 registros: 4 ficheros normales y 60 ficheros con fallo. El fichero resultante contiene 65 líneas incluyendo la cabecera.

5.8. Ejecutor de perfilado

Se ha implementado el perfilador determinista ‘data_profiler.py’. Este ejecutor lee el manifiesto, inspecciona los ficheros ‘.mat’ y genera:

```
codigo/data/interim/cwru_bearing/profile.json
```

El perfil contiene metadatos y estadísticas por canal: número de muestras, valores no finitos, mínimo, máximo, media, desviación típica, RMS, curtosis y energía. No almacena arrays ni señales completas.

La ejecución real ha perfilado 64 ficheros. El resumen agregado contiene 4 ficheros normales, 60 ficheros con fallo, frecuencias de muestreo de 48 kHz y 12 kHz, y los canales detectados ‘BA_time’, ‘DE_time’, ‘FE_time’ y ‘RPM’.

5.9. Ejecutor de Limpieza

Se ha implementado el ejecutor determinista ‘cleaning.py’. Este ejecutor lee el manifiesto y el perfil, extrae el canal principal configurado, elimina valores no finitos, remuestrea la señal a 12 kHz cuando procede y guarda una señal limpia comprimida por fichero:

```
codigo/data/processed/cwru_bearing/clean_signals/
```

También genera un resumen de auditoría en:

```
codigo/data/processed/cwru_bearing/cleaning_summary.json
```

La ejecución real ha producido 64 ficheros ‘.npz’ comprimidos. Los cuatro ficheros normales se han remuestreado de 48 kHz a 12 kHz; los ficheros de fallo ya estaban a 12 kHz. El estado del grafo conserva únicamente la ruta del directorio limpio y las referencias a artefactos, no las señales.

5.10. Ejecutor de Estructuración Temporal

Se ha implementado el ejecutor determinista ‘structuring.py’. Este ejecutor lee las señales limpias, valida que el canal y la frecuencia coincidan con la configuración, genera ventanas temporales con tamaño 2048 y solapamiento del 50 %, calcula features temporales iniciales y crea particiones reproducibles a nivel de fichero.

```
codigo/data/tensors/cwru_bearing/windows_features.csv
codigo/data/tensors/cwru_bearing/windows_raw.npz
codigo/data/tensors/cwru_bearing/splits.json
```

La estrategia de partición evita mezclar ventanas de un mismo fichero entre subconjuntos. Las ventanas normales se reparten entre entrenamiento, validación y test, mientras que las ventanas con fallo se reservan para test. La ejecución real ha generado 7460 ventanas de 2048 muestras: 175 de entrenamiento, 117 de validación y 7168 de test. El tensor crudo tiene forma ‘(7460, 2048)’ y el directorio de tensores ocupa aproximadamente 22 MB.

5.11. Ejecutor de Modelado Base

Se ha implementado el ejecutor determinista ‘modeling.py’ con el primer modelo base de detección de anomalías: Isolation Forest. El modelo se entrena únicamente con ventanas normales del subconjunto de entrenamiento y consume las features temporales generadas por la fase anterior.

```
codigo/models/cwru_bearing/isolation_forest.joblib
codigo/models/cwru_bearing/predictions.csv
codigo/models/cwru_bearing/modeling_summary.json
```

El artefacto ‘joblib’ conserva el modelo entrenado, las columnas usadas y el umbral de decisión. Las predicciones guardan, por cada ventana, identificador, fichero original, partición, etiqueta real, puntuación de anomalía, umbral y predicción binaria. Las métricas finales no se calculan en este ejecutor, sino que se reservan para la fase de evaluación.

La ejecución real ha entrenado el modelo con 175 ventanas normales y ha generado 7460 predicciones. El umbral estimado ha sido 0.5992 y el resumen de modelado registra 398 ventanas predichas como normales y 7062 como anómalas. Estos conteos son una salida preliminar del modelo, no una evaluación final de rendimiento.

5.12. Ejecutor de Evaluación

Se ha implementado el ejecutor determinista ‘evaluation.py’. Este módulo lee las predicciones del modelo, compara la etiqueta binaria real con la predicción de anomalía y calcula métricas técnicas por partición. La partición principal para el MVP es ‘test’.

```
codigo/reports/cwru_bearing/evaluation/metrics.json
codigo/reports/cwru_bearing/evaluation/evaluation_summary.md
```

La ejecución real sobre el split de test ha obtenido precision 0.9991, recall 1.0000, F1-score 0.9996, ROC-AUC 1.0000, PR-AUC 1.0000 y una tasa de falsos positivos de 0.0513. La matriz de confusión ha sido: 111 verdaderos negativos, 6 falsos positivos, 0 falsos negativos y 7051 verdaderos positivos.

Estos resultados deben interpretarse como validación del pipeline y del baseline inicial sobre un benchmark controlado. No constituyen todavía una garantía de generalización industrial, que se deberá analizar con validaciones más exigentes y, posteriormente, con otros datasets como NASA IMS.

5.13. Grafo LangGraph supervisado

Se ha implementado el primer grafo LangGraph del MVP en ‘pipeline.py’ y se ha incorporado un supervisor en el modulo:

```
codigo/app/agents/supervisor.py
```

La version actual ya contempla dos modos: un supervisor LLM que emite una ‘SupervisorDecision’ validada por Pydantic y una politica determinista de respaldo para pruebas, ejecuciones locales sin modelo y recuperacion ante respuestas invalidas.

El grafo ya no avanza directamente de un ejecutor al siguiente. Todas las transiciones vuelven al supervisor:

```
START
-> supervisor
-> manifest_executor
-> supervisor
-> profiler_executor
-> supervisor
-> cleaner_agent
-> supervisor
-> cleaning_executor
-> supervisor
-> structuring_agent
-> supervisor
-> structuring_executor
-> supervisor
-> modeling_agent
-> supervisor
-> modeling_executor
-> supervisor
-> evaluator
-> supervisor
-> evaluation_agent
-> supervisor
-> report_writer
-> supervisor
-> END
```

El grafo valida el estado entrante y saliente con ‘TFMStateModel’, registra mensajes de trazabilidad, acumula referencias a artefactos y detiene la ejecucion de forma

controlada si un ejecutor devuelve estado ‘failed’. Cada decision del supervisor se almacena en ‘messages’ con rol ‘supervisor’ y contenido JSON serializado. El servicio LLM comun vive en:

```
codigo/app/services/llm.py
```

La integracion inicial usa un cliente JSON para Ollama configurable por variables de entorno. Si se activa ‘TFM_SUPERVISOR_MODE=llm’, el supervisor solicita al modelo una decision estructurada; si no hay modelo disponible o la respuesta no valida, se conserva la decision determinista como fallback.

Incluso cuando decide el LLM, la decision queda acotada por validaciones posteriores: la fase actual debe coincidir con el estado real, no se aceptan nodos fuera del plan del MVP, la revision humana permanece desactivada en esta fase local y cualquier salto invalido fuerza el fallback. De este modo se introduce toma de decisiones agentica sin permitir que el modelo ejecute codigo ni manipule datos directamente.

5.14. Agente Limpiador LLM

Se ha implementado el primer agente LLM especializado en:

```
codigo/app/agents/cleaner.py
```

El agente limpiador lee un resumen ligero del perfil del dataset y devuelve una ‘CleaningDecision’ validada por Pydantic. Su decision queda guardada como ‘cleaning_config’ en el estado, pero la transformacion real de las senales sigue delegada exclusivamente en ‘codigo/app/executors/cleaning.py’.

La fase de limpieza queda dividida en decision y ejecucion:

```
supervisor
-> cleaner_agent
-> supervisor
-> cleaning_executor
```

Con Ollama local y el modelo ‘qwen3.5:4b’, se han validado respuestas reales para el supervisor y para el limpiador. El limpiador ha propuesto ‘strategy_id = cwru_clean_llm_v1’, eliminacion de no finitos, remuestreo a 12 kHz y normalizacion ‘none’, cumpliendo el contrato ‘CleaningDecision’.

5.15. Agente Estructurador LLM

Se ha implementado el agente estructurador LLM en:

```
codigo/app/agents/structurer.py
```

Este agente decide la estructuracion temporal del dataset: tamano de ventana, solapamiento, canal principal, frecuencia objetivo, modo de etiquetas, features temporales y rutas esperadas de salida. Su salida principal es una ‘StructuringDecision’ validada por Pydantic. Igual que en el resto de fases, el agente no genera ventanas ni tensores; la ejecucion real permanece en ‘codigo/app/executors/structuring.py’.

La fase de estructuracion queda dividida en decision y ejecucion:

```

supervisor
-> structuring_agent
-> supervisor
-> structuring_executor

```

Con ‘qwen3.5:4b’, el estructurador ha generado una decision valida con ventanas de 2048 muestras, solapamiento del 50 %, canal ‘DE_time’, frecuencia de 12 kHz, modo ‘binary_anomaly’ y features temporales trazables como RMS, curtosis, crest factor y energia.

Durante la integracion se ha detectado que ‘checkpoint_id’ es un nombre reservado por LangGraph 1.1. Por ello se ha retirado de la vista ‘TypedDict’ que circula por el grafo. La persistencia futura debera apoyarse en los checkpoints propios de LangGraph o en un campo de aplicacion que no colisione con nombres reservados.

5.16. Agente Modelador LLM

Se ha implementado el agente modelador LLM en:

```
codigo/app/agents/modeler.py
```

Este agente decide el modelo base y sus hiperparametros a partir del estado ligero del pipeline, la configuracion de estructuracion y un resumen de la tabla de features. Su salida principal es una ‘ModelingDecision’ validada por Pydantic. El agente no entrena modelos ni genera predicciones; la ejecucion real se mantiene en ‘codigo/app/executors/modeling.py’.

La fase de modelado queda dividida en decision y ejecucion:

```

supervisor
-> modeling_agent
-> supervisor
-> modeling_executor

```

Aunque el contrato ‘ModelingConfig’ contempla algoritmos futuros como One-Class SVM, LOF, PCA o autoencoders, el MVP restringe al agente a ‘isolation_forest’, que es el unico algoritmo soportado por el ejecutor determinista actual. Tambien se acotan la semilla (‘random_state = 42’), el split de entrenamiento (‘train’), el uso del split de validacion para el umbral y los hiperparametros permitidos, evitando que el LLM seleccione opciones que la aplicacion aun no puede ejecutar de forma reproducible.

Con ‘qwen3.5:4b’, el modelador ha generado una decision valida con ‘model_name = isolation_forest’, ‘random_state = 42’, entrenamiento sobre el split ‘train’, validacion sobre ‘validation’, ‘n_estimators = 200’, ‘n_jobs = 1’ y ‘threshold_quantile = 0.99’.

Durante esta validacion se ha ajustado el cliente de Ollama para usar ‘think=false’ por defecto en llamadas JSON estrictas, ya que este modelo puede devolver el razonamiento en el campo ‘thinking’ y dejar vacio ‘message.content’ si no se desactiva ese modo. Esta opcion no queda fijada de forma rigida: puede activarse u omitirse mediante:

```

TFM_LLM_THINK=true
TFM_LLM_THINK=auto

```

5.17. Agente Evaluador LLM

Se ha implementado el agente evaluador LLM en:

```
codigo/app/agents/evaluator.py
```

Este agente interpreta las metricas calculadas por el ejecutor determinista de evaluacion y devuelve una ‘EvaluationDecision’ validada por Pydantic. La separacion es deliberada: ‘codigo/app/executors/evaluation.py’ calcula precision, recall, F1, ROC-AUC, PR-AUC, tasa de falsos positivos y matriz de confusion; el agente solo emite el juicio tecnico, la aprobacion y las limitaciones del experimento.

La fase de evaluacion queda dividida en calculo y juicio agentico:

```
supervisor
-> evaluator
-> supervisor
-> evaluation_agent
-> supervisor
```

Para el MVP CWRU se han fijado umbrales conservadores y explicitos:

```
min_recall_required = 0.90
max_false_positive_rate = 0.10
```

El LLM puede redactar el resumen tecnico y las limitaciones, pero no puede aprobar una ejecucion que incumpla esos umbrales. Si la respuesta no valida o contradice las metricas, se activa un fallback determinista.

Tras esta decision, si ‘next_action = continue’, el supervisor ya no finaliza directamente la ejecucion: enruta la fase final hacia el agente redactor. Si el evaluador rechaza la ejecucion, el supervisor termina en ‘failed’ con el resumen de la evaluacion como motivo de parada. El fichero de metricas se genera en:

```
codigo/reports/cwru_bearing/evaluation/metrics.json
```

5.18. Agente Redactor LLM

Se ha implementado el agente redactor LLM en:

```
codigo/app/agents/report_writer.py
```

Este agente decide la estructura del informe tecnico final a partir del estado ligero del pipeline, las metricas, la evaluacion y las referencias a artefactos. Su salida principal es una ‘ReportDecision’ validada por Pydantic. El agente no escribe el informe directamente ni modifica resultados previos: la generacion material del documento se delega en el ejecutor determinista:

```
codigo/app/executors/reporting.py
```

La fase final queda dividida en decision y materializacion del informe:


```
supervisor
-> report_writer
-> supervisor
-> END
```

Para el MVP CWRU, el contrato acota la salida a formato Markdown y a la ruta:

```
codigo/reports/cwru_bearing/final_report.md
```

Tambien se exigen secciones minimas: resumen ejecutivo, contexto y datos, configuraciones del pipeline, metricas y evaluacion, artefactos generados y limitaciones. Las rutas fuente propuestas por el LLM deben existir previamente en el estado como manifiesto, perfil, datos procesados, tensores, metricas o artefactos registrados. Si la respuesta no valida, se aplica un fallback determinista que genera un informe reproducible.

El resultado del ejecutor es un ‘ReportExecutorResult’ y un artefacto de tipo ‘report’. Con ello el pipeline cierra el ciclo completo del MVP: datos crudos, perfilado, limpieza, estructuracion, modelado, evaluacion, juicio agentic e informe tecnico trazable.

5.19. Persistencia Local de Ejecuciones

Como primer hito de la segunda fase se ha implementado una persistencia local de ejecuciones en:

```
codigo/app/services/run_persistence.py
```

El objetivo es conservar una evidencia ligera y auditable de cada ejecucion sin introducir todavia una base de datos externa. El servicio recibe un ‘TFMStateModel’ final y genera una carpeta por ‘run_id’ con el estado final, las decisiones extraidas de los mensajes del grafo, los artefactos, las metricas, la evaluacion, un resumen Markdown y una metadata del snapshot.

```
codigo/reports/runs/<run_id>/state_final.json
codigo/reports/runs/<run_id>/decisions.json
codigo/reports/runs/<run_id>/artifacts.json
codigo/reports/runs/<run_id>/metrics.json
codigo/reports/runs/<run_id>/evaluation.json
codigo/reports/runs/<run_id>/summary.md
codigo/reports/runs/<run_id>/snapshot.json
```

Ademas, se mantiene un indice local:

```
codigo/reports/runs/index.json
```

Esta decision mantiene separado el pipeline puro de la persistencia. La funcion ‘run_cwru_pipeline’ no se modifica; la integracion automatica se realiza mediante un wrapper de ejecucion persistida. De este modo se conserva la estabilidad del grafo ya validado y se prepara la comparacion de ejecuciones para los experimentos de la Fase 2.

5.20. Wrapper de Ejecucion Persistida

El segundo hito de la Fase 2 incorpora el wrapper:

```
run_and_persist_cwru_pipeline(...)
```

Esta funcion ejecuta primero el grafo CWRU con ‘run_cwru_pipeline’, valida el estado final y despues invoca ‘save_run_snapshot’. El resultado se devuelve como un objeto ‘PersistedPipelineRun’, compuesto por el estado final y la metadata del snapshot persistido.

La decision de implementarlo como wrapper, y no como nodo adicional del grafo, mantiene dos rutas claramente separadas: una ejecucion pura del pipeline y una ejecucion con persistencia automatica. Esto evita cambiar el comportamiento ya validado del MVP y permite que los tests sigan inyectando ejecutores y agentes simulados.

La validacion real con CWRU genera una carpeta en:

```
codigo/reports/runs/<run_id>/
```

con 13 artefactos, 17 decisiones y ningun error para una ejecucion completada. Este punto deja preparada la siguiente fase tecnica: construir un registro consultable y comparaciones entre ejecuciones persistentes.

5.21. Registro Consultable de Ejecuciones

El tercer hito de la Fase 2 introduce una capa de consulta sobre las ejecuciones persistentes:

```
codigo/app/services/run_registry.py
```

Este servicio lee el indice local de ejecuciones y los snapshots generados por la persistencia. Permite listar runs, filtrar por dataset, estado final o aprobacion, cargar la metadata de una ejecucion, recuperar sus artefactos y comparar metricas principales entre dos o mas ejecuciones.

La comparacion se devuelve mediante contratos Pydantic propios: ‘RunComparison’, ‘RunComparisonRow’ y ‘MetricComparison’. Para precision, recall y F1-score se considera mejor el valor mas alto, mientras que para la tasa de falsos positivos se considera mejor el valor mas bajo. Esta distincion permite comparar experimentos con criterio tecnico coherente con el dominio industrial.

El registro no recalcula metricas ni vuelve a ejecutar el pipeline; solo consulta evidencia ya persistida. Esta separacion prepara la futura API minima, que podra exponer consultas sobre ejecuciones sin acoplarse a LangGraph ni a los ejecutores.

5.22. Protocolo Experimental Local

El cuarto hito de la Fase 2 incorpora un servicio para ejecutar planes experimentales locales:

```
codigo/app/services/experiment_protocol.py
```

Este servicio define contratos ‘ExperimentPlan’, ‘ExperimentSpec’, ‘ExperimentRunSummary’ y ‘ExperimentPlanResult’. A partir de un plan validado, ejecuta varios runs mediante el wrapper persistido, guarda los artefactos de modelado, evaluacion e informe en carpetas separadas por experimento y genera una comparacion reutilizando ‘run_registry.py’.

El primer plan real, ‘cwru_iforest_threshold_v1’, compara el baseline de Isolation Forest con una variante que solo modifica ‘threshold_quantile’ de 0.99 a 1.00. Los resultados se guardan en:

```
codigo/experiments/cwru_local/cwru_iforest_threshold_v1/
```

La salida incluye ‘experiment_plan.json’, ‘comparison.json’ y ‘results_table.md’. Esta estructura permite llevar resultados comparables a la memoria sin recalcular metricas ni mezclar artefactos de diferentes ejecuciones.

5.23. API Minima de Consulta

El sexto hito de la Fase 2 incorpora una primera API FastAPI sobre la persistencia local:

```
codigo/app/api/app.py
codigo/app/api/routes.py
```

La API se crea mediante ‘create_app(runs_dir=...)’, de modo que en ejecucion real consulta ‘codigo/reports/runs/’ y en tests puede trabajar contra un directorio temporal. Esta primera version es deliberadamente de lectura: permite consultar runs persistidos, recuperar su metadata, listar artefactos y servir el informe Markdown asociado. Tambien expone la comparacion de ejecuciones ya persistidas, reutilizando el registro consultable sin recalcular metricas ni relanzar el pipeline.

Los endpoints iniciales son:

```
GET /health
GET /runs
GET /runs/compare
GET /runs/{run_id}
GET /runs/{run_id}/artifacts
GET /runs/{run_id}/report
```

El endpoint ‘POST /runs’ queda pospuesto hasta definir politicas de ejecucion, aprobacion y control de coste. Esta decision mantiene separada la capa de consulta de la logica que podria lanzar trabajos de modelado o experimentos multiples.

5.24. Dependencias

Se ha creado ‘requirements.txt’ como punto inicial para construir el entorno de Miniconda ‘tfm_v2’, fijado sobre Python 3.11. Incluye dependencias de contratos y grafo, procesamiento de datos, modelos clásicos, API local con FastAPI, visualización técnica y verificación.

5.25. Verificación Actual

La verificación de los contratos se ha realizado con tests unitarios en el entorno objetivo:

```
conda run -n tfm_v2 python -m unittest discover codigo/tests
```

Las pruebas validan la creación del estado inicial CWRU, serialización JSON, rechazo de campos no declarados, restricciones de configuraciones y métricas, coherencia del manifiesto, decisiones de agentes, resultados de ejecutores y generación del manifiesto, perfilado, limpieza, estructuración temporal y modelado base y evaluación CWRU. También validan los adaptadores de entrada para formatos ‘.mat’ y ‘.csv’, y la orquestación del grafo mínimo con ejecutores simulados. También validan el supervisor LLM con cliente fake, el agente limpiador LLM, el agente estructurador LLM, el agente modelador LLM, el agente evaluador LLM, el agente redactor LLM, el ejecutor de informes, el parser JSON, la persistencia local de ejecuciones, el registro consultable, el protocolo experimental local, la API mínima de consulta, la configuración de razonamiento de Ollama, el rechazo de transiciones inválidas y el fallback determinista. En esta fase se han ejecutado 110 tests correctamente.

5.26. Siguiente Paso de Implementación

Tras incorporar la consulta y comparación de runs persistidos por API, el siguiente avance técnico no debería ser lanzar ejecuciones desde HTTP, sino definir primero las políticas de ‘POST /runs’: validación de solicitudes, control de coste, límites de recursos y relación con una futura revisión humana. El punto Human Review queda pospuesto hasta disponer de una aplicación o interfaz donde la aprobación pueda diseñarse con una lógica más ajustada al uso real del sistema.

Capítulo 6

Experimentos

6.1. Protocolo experimental

La Fase 2 introduce un protocolo experimental local para pasar de una unica ejecucion valida a un conjunto pequeno de ejecuciones comparables. El protocolo se implementa en:

```
codigo/app/services/experiment_protocol.py
```

La unidad principal es un manifiesto ‘ExperimentPlan’, compuesto por varias entradas ‘ExperimentSpec’. Cada entrada fija un ‘run_id’, una descripcion y una configuracion de modelado validada mediante ‘ModelingConfig’. La ejecucion del plan reutiliza el wrapper persistido del pipeline y, al finalizar, invoca el registro consultable para comparar metricas entre runs.

Cada plan conserva tres artefactos agregados:

- ‘experiment_plan.json’, con el manifiesto experimental;
- ‘comparison.json’, con la comparacion estructurada;
- ‘results_table.md’, con la tabla resumida para documentacion.

Ademas, los modelos, predicciones, metricas e informes finales se escriben dentro de la carpeta de cada experimento para evitar sobrescrituras entre runs comparables.

6.2. Plan CWRU Isolation Forest Threshold

El primer plan experimental real se identifica como:

```
cwru_iforest_threshold_v1
```

El objetivo es estudiar una variacion controlada del umbral de decision del modelo base Isolation Forest. No se introduce ningun algoritmo nuevo, ya que la regla de la arquitectura exige que todo modelo disponible para un agente tenga un ejecutor determinista reproducible.

Los dos runs ejecutados han sido:

- ‘cwru_iforest_threshold_v1_baseline_threshold_099’;

- ‘cwru_iforest_threshold_v1_conservative_threshold_100’.

Ambos mantienen constante el dataset, la limpieza, la estructuración temporal, el algoritmo, la semilla y ‘n_estimators=200’. La única variable modificada es ‘threshold_quantile’: 0.99 para el baseline y 1.00 para la variante conservadora.

Tabla 6.1: Comparación local de umbral con Isolation Forest sobre CWRU

Experimento	Umbral	Precision	Recall	F1	FPR
Baseline	0.99	0.9991	1.0000	0.9996	0.0513
Conservador	1.00	0.9996	1.0000	0.9998	0.0256

Los dos runs quedan aprobados por el agente evaluador. La variante conservadora reduce la tasa de falsos positivos de 0.0513 a 0.0256 y mejora ligeramente la precisión y el F1-score, manteniendo recall 1.0000. En este benchmark, por tanto, el umbral conservador resulta preferible para reducir falsas alarmas sin penalizar la detección de fallos.

Los artefactos agregados del plan quedan en:

`codigo/experiments/cwru_local/cwru_iforest_threshold_v1/`

6.3. Experimentos posteriores

Experimentos previstos:

- sensibilidad al tamaño de ventana;
- comparación entre features temporales y frecuenciales;
- validación leave-one-load-out;
- validación sobre NASA IMS cuando el MVP esté estabilizado.

Capítulo 7

Resultados

7.1. Resultados cuantitativos

La Fase 2 incorpora una comparacion local entre dos ejecuciones persistidas del baseline Isolation Forest sobre CWRU. Ambas ejecuciones mantienen constante el dataset, la limpieza, la estructuracion temporal, la semilla y ‘n_estimators=200’; la unica variable modificada es el cuantil usado para fijar el umbral de anomalia.

Experimento	Umbral	Precision	Recall	F1	FPR
Baseline	0.99	0.9991	1.0000	0.9996	0.0513
Conservador	1.00	0.9996	1.0000	0.9998	0.0256

Tabla 7.1: Comparacion de umbral con Isolation Forest en el split de test CWRU.

La variante conservadora reduce los falsos positivos sin penalizar el recall. En el baseline, la matriz de confusion del split de test contiene 111 verdaderos negativos, 6 falsos positivos, 0 falsos negativos y 7051 verdaderos positivos. En la variante con umbral 1.00, los falsos positivos bajan a 3, con 114 verdaderos negativos, 0 falsos negativos y 7051 verdaderos positivos.

7.2. Análisis cualitativo

El resultado sugiere que, en este benchmark controlado, un umbral mas conservador es preferible para reducir falsas alarmas manteniendo la deteccion de fallos. La mejora debe interpretarse con cautela, porque CWRU presenta condiciones experimentales limpias y diferencias claras entre estado normal y fallo. La comparacion resulta util como validacion del protocolo experimental y de la trazabilidad de runs, no como evidencia definitiva de generalizacion industrial.

7.3. Limitaciones observadas

Cada resultado deberá acompañarse de limitaciones, supuestos y riesgos de generalización industrial.

Los resultados iniciales se han obtenido sobre CWRU Bearing Dataset, un entorno controlado y con diferencias claras entre condiciones normales y de fallo. Por ello, las

métricas deben interpretarse como una validación del pipeline y del baseline, no como una garantía de rendimiento en escenarios industriales no vistos.

Capítulo 8

Conclusiones

8.1. Conclusiones principales

Esta sección resumirá las aportaciones del proyecto, separando los resultados del pipeline, la arquitectura multiagente y los modelos de detección de anomalías.

8.2. Trabajo futuro

Líneas previstas:

- ampliar la validación con NASA IMS Bearing Dataset;
- incorporar autoencoders para ventanas crudas;
- añadir persistencia de checkpoints;
- exponer el MVP mediante API;
- preparar ejecución Docker y SLURM cuando el flujo local sea estable.

Bibliografía

- [1] Case Western Reserve University Bearing Data Center. 12k drive end bearing fault data, 2026. Disponible en: <https://engineering.case.edu/bearingdatacenter/12k-drive-end-bearing-fault-data>.
- [2] Case Western Reserve University Bearing Data Center. Bearing data center, 2026. Disponible en: <https://engineering.case.edu/bearingdatacenter/welcome>.
- [3] Case Western Reserve University Bearing Data Center. Normal baseline data, 2026. Disponible en: <https://engineering.case.edu/bearingdatacenter/normal-baseline-data>.
- [4] NASA Prognostics Center of Excellence. Data set repository, 2026. Disponible en: <https://www.nasa.gov/intelligent-systems-division/discovery-and-systems-health/pcoe/pcoe-data-set-repository/>.
- [5] NASA Prognostics Center of Excellence. Ims bearings dataset, 2026. Disponible en: <https://phm-datasets.s3.amazonaws.com/NASA/4.+Bearings.zip>.